



TP4 - 3G & LTE

TSM - Advanced Communication Architecture

Group AR-2 : Dimitri Julmy, Dylan Flotte

-

University of Applied Sciences and Arts of Western Switzerland (HES-SO)
Master of Science in Engineering (MSE)
Information and cyber security (ICS)

Repository URI	https://gitlab-url.com
Creation date	2026-03-04
Submission date	2026-03-18

Introduction

Le présent rapport expose les travaux réalisés dans le cadre du laboratoire « TP4 - 3G & LTE », effectué pour le module TSM - Advanced Communication Architecture. L'objectif principal de cette manipulation est de comprendre et d'implémenter les mécanismes d'authentification mutuelle introduits dans les réseaux 3G (UMTS) et perfectionnés dans les réseaux 4G (LTE), en développant un système client-serveur simulant les protocoles de sécurité bidirectionnels.

Les étapes essentielles de l'implémentation sont suivies, depuis la génération du challenge aléatoire (RAND) jusqu'à l'établissement d'une session sécurisée avec authentification mutuelle des deux parties. Ce travail reprend les fondements du GSM tout en introduisant une amélioration majeure : l'authentification du réseau par le terminal mobile via un token cryptographique. L'architecture développée permet ainsi de pallier les vulnérabilités du GSM en garantissant l'identité du serveur, prévenant ainsi les attaques de type IMSI catcher et les faux relais.

Ce laboratoire offre une approche pratique de l'évolution des mécanismes de sécurité entre les générations de réseaux mobiles et permet d'appréhender les améliorations successives introduites par les standards 3G et 4G en matière d'authentification et de protection de la confidentialité des communications.

Implémentation

Code Python

config.py

Ce script contient les paramètres de configuration globaux utilisés par le client et le serveur, notamment les paramètres de connexion, la clé secrète Ki, les chemins des fichiers échangés, et les options de debug.

```
import os

# Paramètres de connexion
HOST = "127.0.0.1"
PORT = 5000

# Clé secrète Ki (stockée dans USIM et AuC)
# En production, cette clé devrait être stockée de manière sécurisée
Ki = b"SuperSecretKey123"

# Durée de vie de la clé de session (20 minutes)
KEY_VALIDITY_SECONDS = 20 * 60

# Répertoire local pour les fichiers échangés
BASE_DIR = os.path.dirname(__file__)
FILES_DIR = os.path.join(BASE_DIR, "files")

# Fichier envoyé par le client vers le serveur
CLIENT_INPUT_FILE = os.path.join(FILES_DIR, "client_payload.txt")

# Fichier sauvegardé côté serveur (reçu du client)
SERVER_RECEIVED_FILE = os.path.join(FILES_DIR, "server_received_from_client.txt")

# Fichier envoyé par le serveur vers le client
SERVER_REPLY_FILE = os.path.join(FILES_DIR, "server_reply.txt")

# Fichier sauvegardé côté client (reçu du serveur)
CLIENT_RECEIVED_FILE = os.path.join(FILES_DIR, "client_received_from_server.txt")

# Paramètres de debug
DEBUG = True
```

crypto.py

Ce script implémente toutes les fonctions cryptographiques nécessaires pour le challenge d'authentification, la dérivation des clés de session, le chiffrement/déchiffrement des données, et la génération des tokens d'authentification.

```
import os
import hashlib
import hmac
import struct
from config import DEBUG

TAG_SIZE = 16
NONCE_SIZE = 8

def generate_rand():
    """
    Génère un nombre aléatoire RAND de 16 octets
    Utilisé par le serveur pour le challenge d'authentification
    """
    rand = os.urandom(16)
    if DEBUG:
        print(f"[CRYPTO] RAND généré: {rand.hex()}")
    return rand

def compute_sres(rand, ki):
    """
    Calcule la réponse signée SRES à partir de RAND et Ki
    SRES = premiers 4 octets de SHA256(RAND || Ki)

    Args:
        rand: Le challenge RAND (16 octets)
        ki: La clé secrète Ki

    Returns:
        SRES (4 octets)
    """
    sres = hashlib.sha256(rand + ki).digest()[:4]
    if DEBUG:
        print(f"[CRYPTO] SRES calculé: {sres.hex()}")
    return sres

def derive_kc(rand, ki):
    """
    Dérive la clé de session Kc à partir de RAND et Ki
    Kc = SHA256(Ki || RAND)

    Args:
        rand: Le challenge RAND (16 octets)
        ki: La clé secrète Ki

    Returns:
        Kc (32 octets)
    """
```

```

"""
kc = hashlib.sha256(ki + rand).digest()
if DEBUG:
    print(f"[CRYPTO] Clé de session Kc dérivée: {kc.hex()}")
return kc

def derive_keystream_keys(kc):
    """
    Dérive des sous-clés dédiées chiffrement et intégrité.

    Args:
        kc: Clé de session principale

    Returns:
        Tuple (k_enc, k_mac)
    """
    k_enc = hmac.new(kc, b"ENC", hashlib.sha256).digest()
    k_mac = hmac.new(kc, b"MAC", hashlib.sha256).digest()
    if DEBUG:
        print("[CRYPTO] Sous-clés dérivées (ENC/MAC)")
    return k_enc, k_mac

def _build_keystream(k_enc, nonce, count, direction, length):
    """
    Génère un flot pseudo-aléatoire avec HMAC-SHA256 en mode compteur.

    Args:
        k_enc: Clé de chiffrement dérivée
        nonce: Nonce aléatoire de 8 octets
        count: Compteur de trame
        direction: 0 (uplink/client->serveur) ou 1 (downlink/serveur->client)
        length: Longueur souhaitée du flot en octets

    Returns:
        Flot de pseudo-aléa de la longueur spécifiée
    """
    if direction not in (0, 1):
        raise ValueError("direction doit valoir 0 (UL) ou 1 (DL)")

    stream = bytearray()
    block_index = 0
    while len(stream) < length:
        material = nonce + struct.pack("!Q", count) + bytes([direction]) + struct.pack("!I",
block_index)
        block = hmac.new(k_enc, material, hashlib.sha256).digest()
        stream.extend(block)
        block_index += 1
    return bytes(stream[:length])

def _xor_bytes(data, key_stream):
    """
    Effectue un XOR octet par octet entre les données et le flot.

    Args:
        data: Données à transformer
        key_stream: Flot pseudo-aléatoire

```

```

Returns:
    Résultat du XOR
    """
    return bytes([d ^ k for d, k in zip(data, key_stream)])

def encrypt_packet(plain_data, k_enc, k_mac, count, direction):
    """
    Chiffre et authentifie un paquet de données.

    Format: NONCE(8) || COUNT(8) || CIPHERTEXT || TAG(16)

    Args:
        plain_data: Données à chiffrer
        k_enc: Clé de chiffrement
        k_mac: Clé d'authentification MAC
        count: Compteur de trame
        direction: 0 (uplink) ou 1 (downlink)

    Returns:
        Paquet chiffré avec tag d'intégrité
        """
    nonce = os.urandom(NONCE_SIZE)
    key_stream = _build_keystream(k_enc, nonce, count, direction, len(plain_data))
    cipher_data = _xor_bytes(plain_data, key_stream)

    mac_input = bytes([direction]) + struct.pack("!Q", count) + nonce + cipher_data
    tag = hmac.new(k_mac, mac_input, hashlib.sha256).digest()[:TAG_SIZE]

    packet = nonce + struct.pack("!Q", count) + cipher_data + tag
    if DEBUG:
        print(f"[CRYPT0] Paquet chiffré: nonce={nonce.hex()} count={count}
    taille={len(packet)}")
    return packet

def decrypt_packet(packet, k_enc, k_mac, direction):
    """
    Vérifie l'intégrité et déchiffre un paquet.

    Args:
        packet: Paquet chiffré au format NONCE || COUNT || CIPHERTEXT || TAG
        k_enc: Clé de chiffrement
        k_mac: Clé d'authentification MAC
        direction: 0 (uplink) ou 1 (downlink)

    Returns:
        Tuple (count, plain_data) - compteur et données déchiffrées
        """
    min_size = NONCE_SIZE + 8 + TAG_SIZE
    if len(packet) < min_size:
        raise ValueError("Paquet trop court")

    nonce = packet[:NONCE_SIZE]
    count = struct.unpack("!Q", packet[NONCE_SIZE:NONCE_SIZE + 8])[0]
    tag = packet[-TAG_SIZE:]
    cipher_data = packet[NONCE_SIZE + 8:-TAG_SIZE]

```

```

mac_input = bytes([direction]) + struct.pack("!Q", count) + nonce + cipher_data
expected_tag = hmac.new(k_mac, mac_input, hashlib.sha256).digest()[TAG_SIZE:]
if not hmac.compare_digest(tag, expected_tag):
    raise ValueError("Tag d'integrite invalide")

key_stream = _build_keystream(k_enc, nonce, count, direction, len(cipher_data))
plain_data = _xor_bytes(cipher_data, key_stream)

if DEBUG:
    print(f"[CRYPTO] Paquet dechiffre: nonce={nonce.hex()} count={count}
taille={len(plain_data)}")
    return count, plain_data

def compute_auth_token(rand, ki):
    """
    Calcule le token d'authentification du serveur
    Token = SHA256(RAND || Ki)

    Args:
        rand: Le challenge RAND
        ki: La clé secrète Ki

    Returns:
        Token d'authentification (32 octets)
    """
    token = hmac.new(ki, rand + b"AUTN", hashlib.sha256).digest()
    if DEBUG:
        print(f"[CRYPTO] Token d'authentification calculé: {token.hex()}")
    return token

```

session.py

Ce script gère l'état d'une session d'authentification, y compris le stockage du RAND, la vérification du SRES, l'établissement de la clé de session Kc, et les fonctions de chiffrement/déchiffrement des données de session. Il encapsule toute la logique liée à une session d'authentification 3G/LTE simulée.

```

import time
from config import Ki, DEBUG, KEY_VALIDITY_SECONDS
from crypto import (
    compute_sres,
    derive_kc,
    derive_keystream_keys,
    encrypt_packet,
    decrypt_packet,
    compute_auth_token,
)

class Session:
    """
    Classe gérant l'état d'une session d'authentification
    """

    def __init__(self, role):
        if role not in ("client", "server"):

```

```

        raise ValueError("role doit valoir 'client' ou 'server'")

    self.role = role
    self.rand = None
    self.kc = None
    self.k_enc = None
    self.k_mac = None
    self.authenticated = False
    self.server_authenticated = False
    self.kc_created_at = None
    self.kc_expires_at = None
    self.tx_count = 0
    self.expected_rx_count = 0
    if DEBUG:
        print(f"[SESSION] Session initialisee (role={self.role})")

def set_rand(self, rand):
    """
    Enregistre le RAND reçu/généré
    """
    self.rand = rand
    if DEBUG:
        print(f"[SESSION] RAND enregistré: {rand.hex()}")

def authenticate_client(self, received_sres):
    """
    Authentifie le client en vérifiant le SRES reçu

    Args:
        received_sres: SRES reçu du client

    Returns:
        True si authentifié, False sinon
    """
    if self.rand is None:
        if DEBUG:
            print("[SESSION] Erreur: RAND non défini")
        return False

    expected_sres = compute_sres(self.rand, Ki)
    self.authenticated = (received_sres == expected_sres)

    if DEBUG:
        if self.authenticated:
            print("[SESSION] Client authentifié avec succès")
        else:
            print("[SESSION] Échec d'authentification du client")
            print(f"[SESSION] SRES attendu: {expected_sres.hex()}")
            print(f"[SESSION] SRES reçu: {received_sres.hex()}")

    return self.authenticated

def authenticate_server(self, received_token):
    """
    Authentifie le serveur en vérifiant le token reçu

    Args:
        received_token: Token reçu du serveur

```



```

Returns:
    True si authentifié, False sinon
"""
if self.rand is None:
    if DEBUG:
        print("[SESSION] Erreur: RAND non défini")
    return False

expected_token = compute_auth_token(self.rand, Ki)
self.server_authenticated = (received_token == expected_token)

if DEBUG:
    if self.server_authenticated:
        print("[SESSION] Serveur authentifié avec succès")
    else:
        print("[SESSION] Échec d'authentification du serveur")
        print(f"[SESSION] Token attendu: {expected_token.hex()}")
        print(f"[SESSION] Token reçu: {received_token.hex()}")

return self.server_authenticated

def establish_session_key(self):
    """
    Établit la clé de session Kc

    Returns:
        La clé de session Kc
    """
    if self.rand is None:
        if DEBUG:
            print("[SESSION] Erreur: RAND non défini")
        return None

    self.kc = derive_kc(self.rand, Ki)
    self.k_enc, self.k_mac = derive_keystream_keys(self.kc)
    self.kc_created_at = time.time()
    self.kc_expires_at = self.kc_created_at + KEY_VALIDITY_SECONDS
    self.tx_count = 0
    self.expected_rx_count = 0
    if DEBUG:
        print(f"[SESSION] Cle de session etablie: {self.kc.hex()}")
        print(f"[SESSION] Cle valide jusqu'a: {time.ctime(self.kc_expires_at)}")
    return self.kc

def _ensure_key_usable(self):
    """
    Vérifie que la clé de session est établie et non expirée.

    Returns:
        True si la clé est utilisable, False sinon
    """
    if self.kc is None or self.k_enc is None or self.k_mac is None:
        if DEBUG:
            print("[SESSION] Erreur: Cle de session non etablie")
        return False

    if time.time() > self.kc_expires_at:
        if DEBUG:
            print("[SESSION] Erreur: Cle de session expiree")

```

```

        return False

    return True

def _tx_direction(self):
    """
    Retourne le type de direction pour l'émission (transmission).

    Returns:
        0 pour uplink (client->serveur), 1 pour downlink (serveur->client)
    """
    return 0 if self.role == "client" else 1

def _rx_direction(self):
    """
    Retourne le type de direction pour la réception.

    Returns:
        1 pour downlink reçu par client, 0 pour uplink reçu par serveur
    """
    return 1 if self.role == "client" else 0

def encrypt_data(self, data):
    """
    Chiffre et authentifie des données avec la clé de session.

    Args:
        data: Données à chiffrer

    Returns:
        Paquet chiffré avec tag d'intégrité, ou None si clé non valid
    """
    if not self._ensure_key_usable():
        return None

    if DEBUG:
        print(f"[SESSION] Chiffrement de {len(data)} octets (count={self.tx_count})")

    packet = encrypt_packet(
        plain_data=data,
        k_enc=self.k_enc,
        k_mac=self.k_mac,
        count=self.tx_count,
        direction=self._tx_direction(),
    )
    self.tx_count += 1
    return packet

def decrypt_data(self, cipher_data):
    """
    Vérifie l'intégrité et déchiffre des données de session.

    Args:
        cipher_data: Paquet chiffré à déchiffrer

    Returns:
        Données déchiffrées, ou None si vérification échoue
    """
    if not self._ensure_key_usable():

```

```

        return None

if DEBUG:
    print(f"[SESSION] Dechiffrement de {len(cipher_data)} octets")

try:
    count, plain_data = decrypt_packet(
        packet=cipher_data,
        k_enc=self.k_enc,
        k_mac=self.k_mac,
        direction=self._rx_direction(),
    )
except ValueError as err:
    if DEBUG:
        print(f"[SESSION] Erreur de dechiffrement: {err}")
    return None

if count != self.expected_rx_count:
    if DEBUG:
        print(
            f"[SESSION] Rejet paquet: count attendu={self.expected_rx_count},
recu={count}"
        )
    return None

self.expected_rx_count += 1
return plain_data

def get_auth_token(self):
    """
    Génère le token d'authentification du serveur

    Returns:
        Token d'authentification
    """
    if self.rand is None:
        if DEBUG:
            print("[SESSION] Erreur: RAND non défini")
        return None

    return compute_auth_token(self.rand, Ki)

def is_authenticated(self):
    """
    Vérifie si la session est authentifiée

    Returns:
        True si authentifié, False sinon
    """
    return self.authenticated

def is_server_authenticated(self):
    """
    Vérifie si le serveur est authentifié

    Returns:
        True si authentifié, False sinon
    """
    return self.server_authenticated

```

```
def is_key_valid(self):
    """
    Vérifie si la clé de session est toujours valide.

    Returns:
        True si la clé n'a pas expiré, False sinon
    """
    if self.kc_expires_at is None:
        return False
    return time.time() <= self.kc_expires_at
```

server.py

Ce script implémente un serveur d'authentification 3G/LTE. Il gère la connexion avec le client, l'authentification mutuelle, l'établissement de la clé de session, et l'échange de données chiffrées avec intégrité.

```
import socket
import os
import struct
from config import HOST, PORT, DEBUG, FILES_DIR, SERVER_RECEIVED_FILE, SERVER_REPLY_FILE
from crypto import generate_rand
from session import Session
```

```
def recv_exact(conn, size):
    """
    Reçoit exactement 'size' octets depuis la connexion.

    Args:
        conn: Socket de connexion
        size: Nombre d'octets à recevoir

    Returns:
        Données brutes reçues
    """
    data = b""
    while len(data) < size:
        chunk = conn.recv(size - len(data))
        if not chunk:
            raise ConnectionError("Connexion interrompue")
        data += chunk
    return data
```

```
def send_frame(conn, payload):
    """
    Envoie un frame: longueur (4 bytes) + payload.

    Args:
        conn: Socket de connexion
        payload: Contenu du frame à envoyer
    """
    conn.sendall(struct.pack("!I", len(payload)) + payload)
```

```

def recv_frame(conn):
    """
    Reçoit un frame: lit d'abord la longueur, puis le contenu.

    Args:
        conn: Socket de connexion

    Returns:
        Contenu du frame reçu
    """
    header = recv_exact(conn, 4)
    length = struct.unpack("!I", header)[0]
    return recv_exact(conn, length)


def pack_file_payload(filename, content):
    """
    Encode un fichier au format: longueur du nom (2 bytes) + nom + contenu.

    Args:
        filename: Nom du fichier
        content: Contenu du fichier en bytes

    Returns:
        Payload encodé prêt à être chiffré/envoyé
    """
    name_bytes = filename.encode("utf-8")
    if len(name_bytes) > 65535:
        raise ValueError("Nom de fichier trop long")
    return struct.pack("!H", len(name_bytes)) + name_bytes + content


def unpack_file_payload(payload):
    """
    Décode un fichier à partir du payload reçu.

    Args:
        payload: Payload encodé au format: longueur nom + nom + contenu

    Returns:
        Tuple (filename, content) - nom et contenu du fichier
    """
    if len(payload) < 2:
        raise ValueError("Payload fichier invalide")
    name_len = struct.unpack("!H", payload[:2])[0]
    if len(payload) < 2 + name_len:
        raise ValueError("Payload fichier tronqué")
    filename = payload[2:2 + name_len].decode("utf-8", errors="replace")
    content = payload[2 + name_len:]
    return filename, content


def main():
    """
    Serveur d'authentification 3G/LTE
    """
    # Initialisation de la session
    session = Session(role="server")

    os.makedirs(FILE_DIR, exist_ok=True)

```

```

# Création du socket serveur
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print(f"[SERVEUR] En attente de connexion sur {HOST}:{PORT}...")
print()

conn, addr = server.accept()
print(f"[SERVEUR] Client connecté depuis: {addr}")
print()

try:
    # ÉTAPE 1: Génération et envoi du RAND
    RAND = generate_rand()
    session.set_rand(RAND)
    send_frame(conn, RAND)
    print(f"[SERVEUR] RAND envoyé au client: {RAND.hex()}")
    print()

    # ÉTAPE 2: Réception et vérification du SRES
    print("[SERVEUR] Attente de la réponse SRES du client...")
    client_sres = recv_frame(conn)
    print(f"[SERVEUR] SRES reçu: {client_sres.hex()}")

    if not session.authenticate_client(client_sres):
        print("[SERVEUR] ÉCHEC: Authentification du client échouée")
        conn.close()
        server.close()
        return

    print("[SERVEUR] Client authentifié avec succès")
    print()

    # ÉTAPE 3: Authentification du serveur auprès du client
    token = session.get_auth_token()
    send_frame(conn, token)
    print(f"[SERVEUR] Token d'authentification envoyé: {token.hex()}")
    print()

    # ÉTAPE 4: Établissement de la clé de session
    Kc = session.establish_session_key()
    print(f"[SERVEUR] Clé de session Kc établie: {Kc.hex()}")
    print()

    # ÉTAPE 5: Réception des données chiffrées
    print("[SERVEUR] Attente des données du client...")
    cipher_data = recv_frame(conn)
    print(f"[SERVEUR] Données chiffrées reçues ({len(cipher_data)} octets): {cipher_data.hex()}")

    plain = session.decrypt_data(cipher_data)
    if plain is None:
        print("[SERVEUR] Erreur: echec de dechiffrement/verification integrite")
        return

    filename, file_content = unpack_file_payload(plain)
    with open(SERVER_RECEIVED_FILE, "wb") as out_file:

```

```

        out_file.write(file_content)

    print(f"[SERVEUR] Fichier reçu: {filename} ({len(file_content)} octets)")
    print(f"[SERVEUR] Fichier sauvegarde: {SERVER_RECEIVED_FILE}")
    print()

    # ÉTAPE 6: Envoi de la réponse chiffrée
    server_text = (
        b"Accuse de reception: fichier client reçu et verifie.\n"
        b"Simulation 3G/LTE: chiffrement flot + integrite HMAC + compteur.\n"
    )
    with open(SERVER_REPLY_FILE, "wb") as reply_file:
        reply_file.write(server_text)

    reply_payload = pack_file_payload(os.path.basename(SERVER_REPLY_FILE), server_text)
    print(f"[SERVEUR] Fichier de reponse a envoyer: {SERVER_REPLY_FILE}")

    cipher = session.encrypt_data(reply_payload)
    if cipher is None:
        print("[SERVEUR] Erreur: cle de session invalide/expiree")
        return

    print(f"[SERVEUR] Message chiffré: {cipher.hex()}")
    send_frame(conn, cipher)
    print("[SERVEUR] Réponse envoyée au client")
    print()

except Exception as e:
    print(f"[SERVEUR] Erreur: {e}")
finally:
    conn.close()
    server.close()
    print("[SERVEUR] Connexion fermée")

if __name__ == "__main__":
    main()

```

client.py

Ce script implémente un client d'authentification 3G/LTE. Il gère la connexion au serveur, l'authentification mutuelle, l'établissement de la clé de session, et l'échange de données chiffrées avec intégrité.

```

import socket
import os
import struct
from config import (
    HOST,
    PORT,
    DEBUG,
    FILES_DIR,
    CLIENT_INPUT_FILE,
    CLIENT_RECEIVED_FILE,
)
from crypto import compute_sres
from session import Session

```

```

def recv_exact(conn, size):
    """
    Reçoit exactement 'size' octets depuis la connexion.

    Args:
        conn: Socket de connexion
        size: Nombre d'octets à recevoir

    Returns:
        Données brutes reçues
    """
    data = b""
    while len(data) < size:
        chunk = conn.recv(size - len(data))
        if not chunk:
            raise ConnectionError("Connexion interrompue")
        data += chunk
    return data


def send_frame(conn, payload):
    """
    Envoie un frame: longueur (4 bytes) + payload.

    Args:
        conn: Socket de connexion
        payload: Contenu du frame à envoyer
    """
    conn.sendall(struct.pack("!I", len(payload)) + payload)


def recv_frame(conn):
    """
    Reçoit un frame: lit d'abord la longueur, puis le contenu.

    Args:
        conn: Socket de connexion

    Returns:
        Contenu du frame reçu
    """
    header = recv_exact(conn, 4)
    length = struct.unpack("!I", header)[0]
    return recv_exact(conn, length)


def pack_file_payload(filename, content):
    """
    Encode un fichier au format: longueur du nom (2 bytes) + nom + contenu.

    Args:
        filename: Nom du fichier
        content: Contenu du fichier en bytes

    Returns:
        Payload encodé prêt à être chiffré/envoyé
    """
    name_bytes = filename.encode("utf-8")

```



```

if len(name_bytes) > 65535:
    raise ValueError("Nom de fichier trop long")
return struct.pack("!H", len(name_bytes)) + name_bytes + content

def unpack_file_payload(payload):
    """
    Décode un fichier à partir du payload reçu.

    Args:
        payload: Payload encodé au format: longueur nom + nom + contenu

    Returns:
        Tuple (filename, content) - nom et contenu du fichier
    """
    if len(payload) < 2:
        raise ValueError("Payload fichier invalide")
    name_len = struct.unpack("!H", payload[:2])[0]
    if len(payload) < 2 + name_len:
        raise ValueError("Payload fichier tronqué")
    filename = payload[2:2 + name_len].decode("utf-8", errors="replace")
    content = payload[2 + name_len:]
    return filename, content

def main():
    """
    Client d'authentification 3G/LTE
    """
    # Initialisation de la session
    session = Session(role="client")

    os.makedirs(FILE_DIR, exist_ok=True)

    # Connexion au serveur
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    print(f"[CLIENT] Connexion au serveur {HOST}:{PORT}...")

    try:
        client.connect((HOST, PORT))
        print("[CLIENT] Connecté au serveur")
        print()

        # ÉTAPE 1: Réception du RAND
        print("[CLIENT] Attente du RAND du serveur...")
        RAND = recv_frame(client)
        session.set_rand(RAND)
        print(f"[CLIENT] RAND reçu: {RAND.hex()}")
        print()

        # ÉTAPE 2: Calcul et envoi du SRES
        print("[CLIENT] Calcul du SRES...")
        from config import Ki
        SRES = compute_sres(RAND, Ki)
        print(f"[CLIENT] SRES calculé: {SRES.hex()}")
        send_frame(client, SRES)
        print("[CLIENT] SRES envoyé au serveur")
        print()
    
```

```

# ÉTAPE 3: Authentification du serveur
print("[CLIENT] Attente du token d'authentification...")
token = recv_frame(client)
print(f"[CLIENT] Token reçu: {token.hex()}")

if not session.authenticate_server(token):
    print("[CLIENT] ÉCHEC: Serveur non authentifié")
    client.close()
    return

print("[CLIENT] Serveur authentifié avec succès")
print()

# ÉTAPE 4: Établissement de la clé de session
Kc = session.establish_session_key()
print(f"[CLIENT] Clé de session Kc établie: {Kc.hex()}")
print()

# ÉTAPE 5: Envoi d'un fichier chiffré
if not os.path.exists(CLIENT_INPUT_FILE):
    default_content = (
        b"Fichier client pour simulation securisee 3G/LTE.\n"
        b"Contenu chiffre avec Kc temporaire, nonce, compteur et tag MAC.\n"
    )
    with open(CLIENT_INPUT_FILE, "wb") as default_file:
        default_file.write(default_content)

with open(CLIENT_INPUT_FILE, "rb") as input_file:
    file_content = input_file.read()

print(f"[CLIENT] Fichier a envoyer: {CLIENT_INPUT_FILE} ({len(file_content)} octets)")
payload = pack_file_payload(os.path.basename(CLIENT_INPUT_FILE), file_content)

cipher = session.encrypt_data(payload)
if cipher is None:
    print("[CLIENT] Erreur: cle de session invalide/expiree")
    return

print(f"[CLIENT] Données chiffrées: {cipher.hex()}")
send_frame(client, cipher)
print("[CLIENT] Données envoyées au serveur")
print()

# ÉTAPE 6: Réception de la réponse
print("[CLIENT] Attente de la réponse du serveur...")
cipher_resp = recv_frame(client)
print(f"[CLIENT] Réponse chiffrée reçue: {cipher_resp.hex()}")

resp = session.decrypt_data(cipher_resp)
if resp is None:
    print("[CLIENT] Erreur: echec de dechiffrement/verification integrite")
    return

reply_name, reply_content = unpack_file_payload(resp)
with open(CLIENT_RECEIVED_FILE, "wb") as received_file:
    received_file.write(reply_content)

print(f"[CLIENT] Fichier reçu du serveur: {reply_name} ({len(reply_content)} octets)")
print(f"[CLIENT] Fichier sauvegarde: {CLIENT_RECEIVED_FILE}")

```

```

print()

except Exception as e:
    print(f"[CLIENT] Erreur: {e}")
finally:
    client.close()
    print("[CLIENT] Connexion fermée")

if __name__ == "__main__":
    main()

```

Capture du fonctionnement

```

[SESSION] Session initialisée
[SERVEUR] En attente de connexion sur 127.0.0.1:5000...

[SERVEUR] Client connecté depuis: ('127.0.0.1', 48060)

[CRYPTO] RAND généré: 264cabd49f335d16f8a2918fe0d18f76
[SESSION] RAND enregistré: 264cabd49f335d16f8a2918fe0d18f76
[SERVEUR] RAND envoyé au client: 264cabd49f335d16f8a2918fe0d18f76

[SERVEUR] Attente de la réponse SRES du client...
[SERVEUR] SRES reçu: c5314078
[CRYPTO] SRES calculé: c5314078
[SESSION] Client authentifié avec succès
[SERVEUR] Client authentifié avec succès

[CRYPTO] Token d'authentification calculé: c5314078ea3ecf1c027ec38093c6f721bba3fb18220f425cb6fd08bdd598c57c
[SERVEUR] Token d'authentification envoyé: c5314078ea3ecf1c027ec38093c6f721bba3fb18220f425cb6fd08bdd598c57c

[CRYPTO] Clé de session Kc dérivée: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6
[SESSION] Clé de session établie: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6
[SERVEUR] Clé de session Kc établie: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6

[SERVEUR] Attente des données du client...
[SERVEUR] Données chiffrées reçues (64 octets): 625887fd1ea4e0da3fc8bb161f5f895bbe942bbd96d1bc44f1acd9ce2e4d0e87335a82f91ea3e7dc3ec8b81c4a5edc5debca7deec281e915a9fd8e9c7a1252d6
[SESSION] Déchiffrement de 64 octets
[CRYPTO] XOR cipher appliqué - Taille: 64 octets
[SERVEUR] Données déchiffrées: b'a3f7b2c1d9e845607f2c1a3b9d4e6f8a01c3b5d7e9f2a4c6b8d0e1f3a5c7b9d0'

[SERVEUR] Message à envoyer: b'Accuse de reception'
[SESSION] Chiffrement de 19 octets
[CRYPTO] XOR cipher appliqué - Taille: 19 octets
[SERVEUR] Message chiffré: 420802bf0ff3a38f3ed1ac4b480fc1fe09d77
[SERVEUR] Réponse envoyée au client

[SERVEUR] Connexion fermée

```

Fig. 1. – Capture fonctionnement côté serveur

```
[SESSION] Session initialisée
[CLIENT] Connexion au serveur 127.0.0.1:5000...
[CLIENT] Connecté au serveur

[CLIENT] Attente du RAND du serveur...
[SESSION] RAND enregistré: 264cabd49f335d16f8a2918fe0d18f76
[CLIENT] RAND reçu: 264cabd49f335d16f8a2918fe0d18f76

[CLIENT] Calcul du SRES...
[CRYPTO] SRES calculé: 7042c7e4
[CRYPTO] SRES calculé: c5314078
[CLIENT] SRES calculé: c5314078
[CLIENT] SRES envoyé au serveur

[CLIENT] Attente du token d'authentification...
[CLIENT] Token reçu: c5314078ea3ecf1c027ec38093c6f721bba3fb18220f425cb6fd08bdd598c57c
[CRYPTO] Token d'authentification calculé: c5314078ea3ecf1c027ec38093c6f721bba3fb18220f425cb6fd08bdd598c57c
[SESSION] Serveur authentifié avec succès
[CLIENT] Serveur authentifié avec succès

[CRYPTO] Clé de session Kc dérivée: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6
[SESSION] Clé de session établie: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6
[CLIENT] Clé de session Kc établie: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6

[CLIENT] Données à envoyer (64 octets): b'a3f7b2c1d9e845607f2c1a3b9d4e6f8a01c3b5d7e9f2a4c6b8d0e1f3a5c7b9d0'
[SESSION] Chiffrement de 64 octets
[CRYPTO] XOR cipher appliqué - Taille: 64 octets
[CLIENT] Données chiffrées: 625887fd1ea4e0da3fc8bb161f5f895bbe942bbd96d1bc44f1acd9ce2e4d0e87335a82f91ea3e7dc3ec8b81c4a5edc5debca7deec281e915a9fd8e9c7a1252d6
[CLIENT] Données envoyées au serveur

[CLIENT] Attente de la réponse du serveur...
[CLIENT] Réponse chiffrée reçue: 420882bf0ff3a38f3ed1ac4b480fcf1fe09d77
[SESSION] Déchiffrement de 19 octets
[CRYPTO] XOR cipher appliqué - Taille: 19 octets
[CLIENT] Réponse déchiffrée: b'Accuse de reception'
```

Fig. 2. – Capture fonctionnement côté client

Conclusion

Ce laboratoire a permis de valider la mise en œuvre complète d'un système d'authentification mutuelle conforme aux principes des réseaux 3G et LTE, confirmant le bon fonctionnement des mécanismes cryptographiques de challenge-réponse bidirectionnels. L'implémentation développée illustre l'évolution significative par rapport au GSM, avec l'introduction d'un token d'authentification permettant au client de vérifier l'identité du serveur, établissant ainsi une session doublement authentifiée avant tout échange de données.

Malgré l'utilisation d'un chiffrement XOR simplifié pour cette simulation, l'architecture mise en place démontre efficacement les principes de l'authentification mutuelle et la dérivation sécurisée de clé de session K_c . Les captures de fonctionnement présentées attestent de la réussite des échanges cryptographiques bidirectionnels et de la validation mutuelle des identités, prévenant ainsi les attaques par usurpation d'identité du réseau.

Enfin, cette étude pratique met en lumière les améliorations apportées par les générations successives de réseaux mobiles, notamment la protection contre les IMSI catchers et les faux relais qui exploitaient l'absence d'authentification du réseau dans le GSM. Elle souligne également la continuité de l'évolution vers le LTE-Advanced et la 5G, qui renforcent encore davantage ces mécanismes avec des algorithmes cryptographiques plus robustes et une meilleure protection de la vie privée. Cette expérience constitue ainsi un maillon essentiel dans la compréhension de l'évolution des architectures de sécurité des communications mobiles.